STAT 387 **EXTRA CREDIT EXERCISE** Winter 2026 INTRODUCTION TO STATISTICAL LEARNING

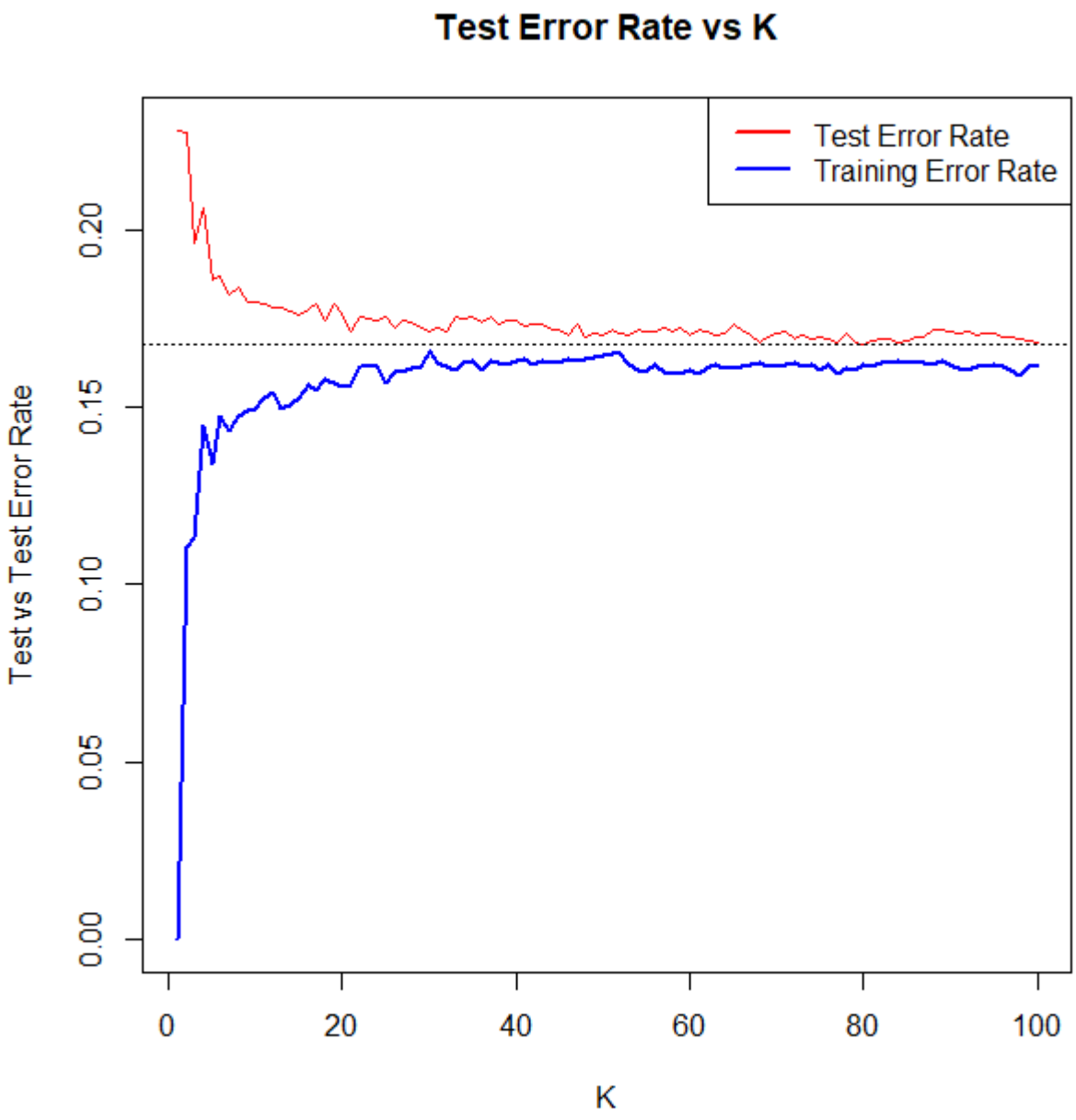
.

1)(a) Fit KNN with *K* = 1, 2, …, 30, …, 35, …, 100.

	K	Train_Error	Test_Error
1	1	0.0000	0.2280
2	2	0.1105	0.2275
3	30	0.1655	0.1715
4	35	0.1630	0.1755
5	100	0.1615	0.1680

1)(b) Plot training and test error rates against *K*. Explain what you observe. Is it consistent with what you expect from the class?

These results are consistent with the Bias-Variance Tradeoff discussed in class. At low values of K the model is highly flexible, leading to a training error of zero but a high test error due to overfitting. As K increases, the model becomes less flexible and more stable; the training error increases while the test error decreases. The error rates eventually stabilize, and the optimal K is found at the point where the test error is minimized. Beyond this point, the model begins to underfit, and both error rates may begin to rise as the decision boundary becomes too simple.

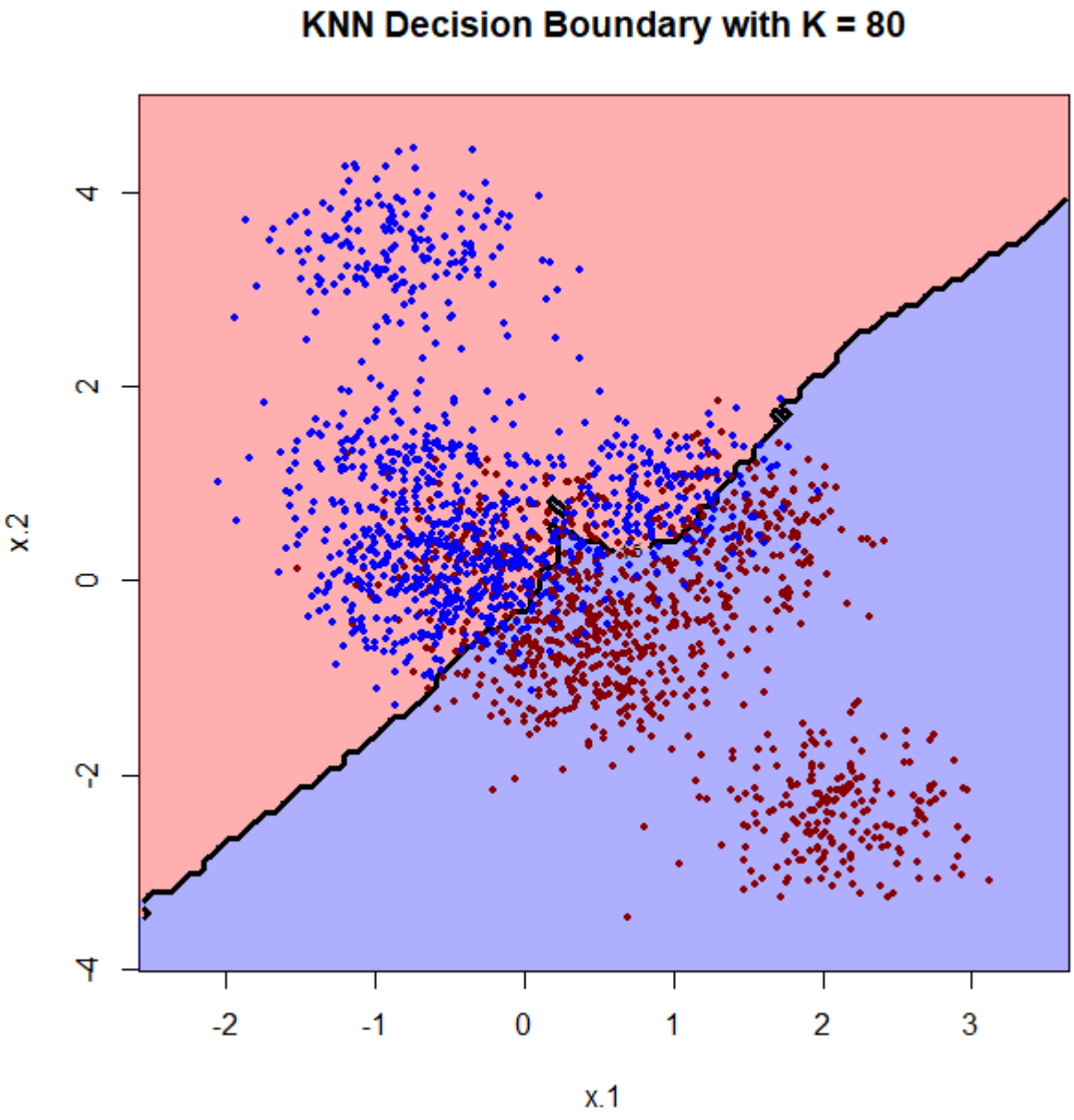

Test Error Rate vs K
Test Error Rate
Training Error Rate
Test vs Test Error Rate
0.00
0.05
0.10
0.15
0.20
0
20
40
60
80
100
K

1)(c) What is the optimal value of *K*? What are the corresponding training and test error rates? Also, what are the corresponding sensitivity and specificity values for test data?

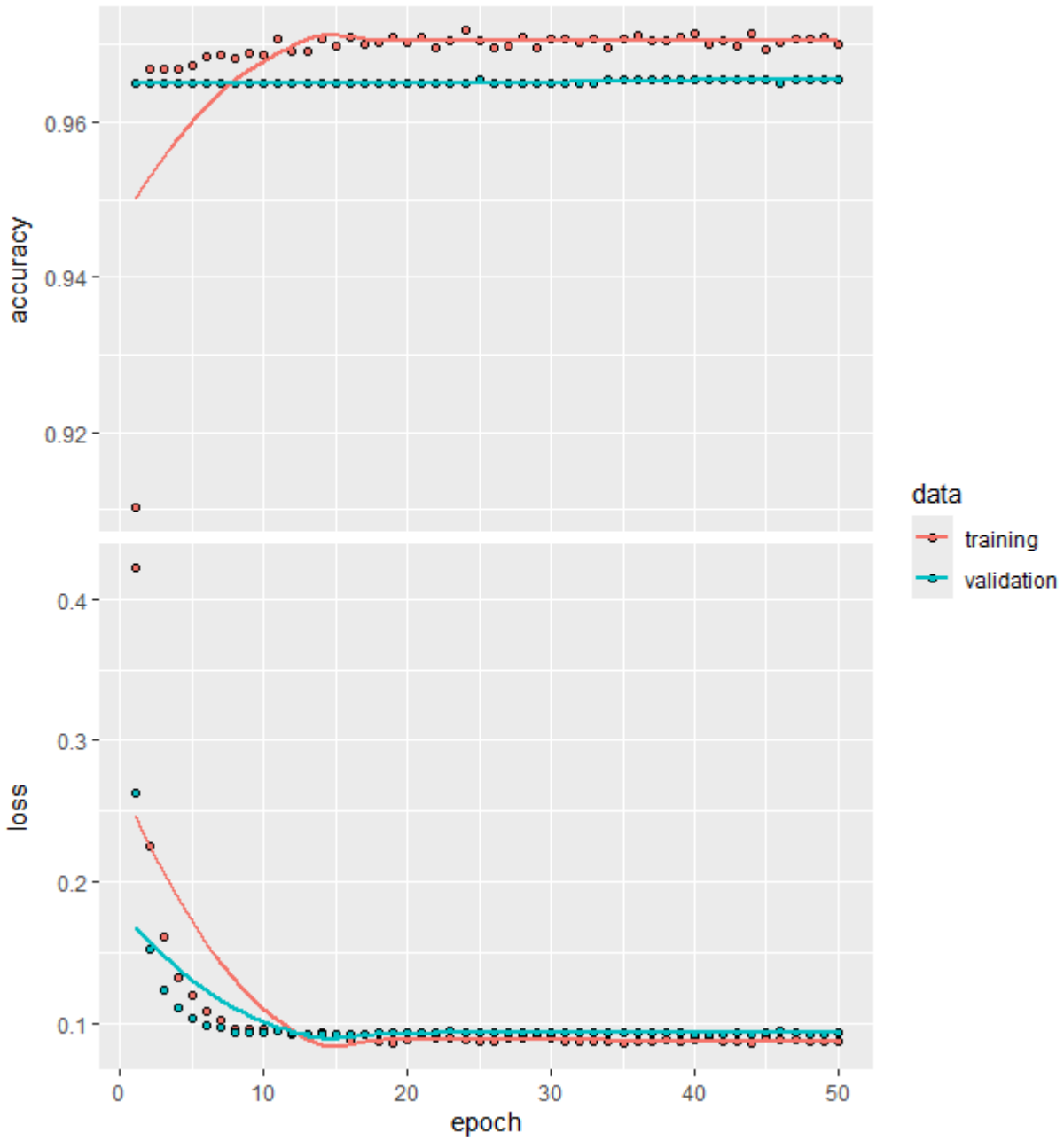
The optimal value for K is identified as 80. This choice yields a training error rate of 0.162 and a test error rate of 0.1675. For the test data, the model achieved a sensitivity of 77.8% (identifying 'yes' cases) and a specificity of 88.7% (identifying 'no' cases).

(d) Make a plot of the training data and superimpose the decision boundary for the optimal *K*. Comment on what you observe. Does the decision boundary seem sensible? Is there a simple way to describe the decision boundary?

Comments: The decision boundary for the optimal K=80 is sensible as it avoids the 'jagged' edges typical of overfitting while still separating the two clusters effectively. It displays a smooth curvature that bisects the overlapping classes, indicating a stable model with low variance. A simple way to describe this boundary is as a nearly linear, diagonal line. This suggests that the relationship between the two features and the classes is largely linear, and a simpler model like Logistic Regression might perform similarly well.


KNN Decision Boundary with K = 80
x.2
x.1
4
2
0
-2
-4
-2
-1
0
1
2
3

2)(a) Fit a neural network using a single hidden layer with 10 units, and dropout regularization. Have a look at *Labs 10.9.1 – 10.9.2* in the textbook for guidance.



2)(b) Compare the classification performance of your model with that of linear logistic

regression.

The GLM model achieves an accuracy of 97.25%, which slightly outperforms the Neural Network's accuracy of 96.55%. While both models show high overall accuracy, a closer look at the confusion matrices reveals a significant difference in how they handle the minority class (confusion matrices are below).

The GLM model was more effective at detecting actual defaults, correctly identifying 17 cases compared to only 3 identified by the Neural Network. This indicates that for this specific imbalanced dataset, the simpler Logistic Regression model is more robust. The Neural Network, despite its complexity and the use of dropout regularization, appears to be biased toward the majority class ('No' defaults), likely because it is optimizing for overall accuracy rather than sensitivity.

And the confusion matrix for the neural network is :

```
       yTest
nnClass    0    1
      0 1930   67
      1    0    3
```

The confusion matrix for the GLM table is:

```
             yTest
logisticClass    0    1
            0 1928   53
            1    2   17
```

R Code:

Question 1:

```r
library(class)
#Loading in the CSV files for the training and test data
training<-read.csv("1-training_data.csv")
test<-read.csv("1-test_data.csv")
set.seed(1)

#Consider the training and test data posted in R Lab under Modules in CANVAS as 1-training-
#data.csv and 1-test-data.csv, respectively, for a two-class classification problem.

#(a) Fit KNN with K = 1, 2, …, 30, …, 35, …, 100.
xTrain<-training[,-3]
yTrain<-training$y
xTest<-test[,-3]
yTest<-test$y
#calculate test error rates
testErrorRates<- sapply(1:100, function(k) {
  pred <- knn(xTrain, xTest, cl=yTrain, k=k)
  mean(pred != yTest)
})
#calculate training error rates
trainingErrorRates <- sapply(1:100, function(k) {
  pred <- knn(xTrain, xTrain, cl=yTrain, k=k)
  mean(pred != yTrain)
})
#plot training and test rates against K, explain what you observe, is it consistent
#with what you expect from the class?
size=range(c(testErrorRates, trainingErrorRates))
plot(1:100, testErrorRates, type="l", col="red",
     xlab="K",
     ylab="Test vs Test Error Rate",
     main="Test Error Rate vs K",
     ylim=size)
lines(1:100, trainingErrorRates, type="l", col="blue", lwd=2)
legend("topright", legend=c("Test Error Rate", "Training Error Rate"),
col=c("red", "blue"), lty=1, lwd=2)
abline(h = min(testErrorRates), col = "black", lty = 3)

#c) What is the optimal value of K? What are the corresponding training and test error rates?
#Also, what are the corresponding sensitivity and specificity values for test data?
optimalK <- which.min(testErrorRates)
optimalK
abline(v=optimalK, col="forestgreen", lwd=2, lty=2)
testErrorRates[optimalK]
trainingErrorRates[optimalK]

#calculate sensitivity and specificity for test data
predOptimal <- knn(xTrain, xTest, cl=yTrain, k=optimalK)
confusionMatrix <- table(predOptimal, yTest)
sensitivity <- confusionMatrix[2, 2] / sum(confusionMatrix[, 2])
specificity <- confusionMatrix[1, 1] / sum(confusionMatrix[, 1])
sensitivity
specificity

#(d) Make a plot of the training data and superimpose the decision boundary for the optimal K.
#Comment on what you observe. Does the decision boundary seem sensible? Is there a
#simple way to describe the decision boundary?
#define the grid
px<-seq(min(training$x.1)-0.5, max(training$x.1)+0.5, length.out=100)
py<-seq(min(training$x.2)-0.5, max(training$x.2)+0.5, length.out=100)
grid= expand.grid(x.1=px, x.2=py)

#predict class labels for grid points based on our selected optimal k
gridPred<- knn(train=training[,-3], test=grid, cl=training$y, k=optimalK)
gridMatrix<-matrix(as.numeric(gridPred), length(px), length(py))
#coloration the background
image(px, py, gridMatrix,
      col=c(rgb(1, 0, 0, alpha=0.3), rgb(0, 0, 1, alpha=0.3)),
      xlab="x.1", ylab="x.2",
      main = paste("KNN Decision Boundary with K =", optimalK))

#superimpose the decision boundary
contour(px, py, gridMatrix, levels=1.5, add=TRUE, lwd=3)

#plot points
points(training$x.1, training$x.2,
     col=ifelse(training$y=="yes", "darkred", "blue"),
     pch=20, cex=0.9)
```

Question 2:

```r
#Consider Default data from the ISLR2 package. We are interested in predicting whether an
#individual will default on their credit card payment or not. The data set is displayed in Figu
#4.1. 80% of the data will be taken as training data.
#(a) Fit a neural network using a single hidden layer with 10 units, and dropout regularization
#Have a look at Labs 10.9.1 - 10.9.2 in the textbook for guidance.
library(ISLR2)
#library(keras)
library(keras3)
library(tensorflow)
data(Default)

summary(Default)
#convert factors to numerics for NN
Default$default <- ifelse(Default$default == "Yes", 1, 0)
Default$student <- ifelse(Default$student == "Yes", 1, 0)

set.seed(1)
training<-sample(1:nrow(Default), size=0.8*nrow(Default))

#scaling model matrix
x<- scale(model.matrix(default~. -1, data=Default))
#force numerics
x<- matrix(as.numeric(x), nrow=nrow(x))
y<- as.numeric(Default$default)

xTrain<-x[training,]
yTrain<-y[training]

xTest<-x[-training,]
yTest<-y[-training]

#build model
model <- keras_model_sequential()

model$add(layer_input(shape = ncol(xTrain)))
model$add(layer_dense(units = 10, activation = "relu"))
model$add(layer_dropout(rate = 0.4))
model$add(layer_dense(units = 1, activation = "sigmoid"))

model|>compile(loss= "binary_crossentropy",
                  optimizer = optimizer_rmsprop(),
                  metrics = list("accuracy"))

history <- model|>fit(xTrain, yTrain,
                        epochs = 50,
                        batch_size = 32,
                        validation_data = list(xTest, yTest))

plot(history)

#linear model vs neural network model comparisons
glmModel <- glm(default ~ ., data=Default[training,], family="binomial")

glmPred <- predict(glmModel, Default[-training,], type="response")
glmClass <- ifelse(glmPred > 0.5, 1, 0)
glmTable <- table(glmClass, yTest)
glmAccuracy <- mean(glmClass == yTest)

nnPred <- model %>% predict(xTest)
nnClass <- ifelse(nnPred > 0.5, 1, 0)
nnTable <- table(nnClass, yTest)
nnAccuracy <- mean(nnClass == yTest)

glmAccuracy
nnAccuracy

glmTable
nnTable

#table of actual values for default
table(Default$default)
```